

```

import { existsSync, readFileSync, writeFileSync } from "node:fs";

// ----- Types -----
interface Listing {
  id: string;
  source: "vinted" | "ebay";
  title: string;
  price: number;
  currency: string;
  url: string;
  image?: string;
  extra?: string;
}

interface Search { name: string; query: string; maxPrice?: number; exclude?: string; }

const config = JSON.parse(readFileSync("config.json", "utf8")) as {
  intervalSeconds: number; exclude: string[]; searches: Search[];
};

const sleep = (ms: number) => new Promise((r) => setTimeout(r, ms));

// ----- Mémoire des annonces déjà vues -----
const SEEN_FILE = "seen.json";
const wasEmpty = !existsSync(SEEN_FILE);
const seen = new Set<string>(wasEmpty ? [] : JSON.parse(readFileSync(SEEN_FILE, "utf8")));
const saveSeen = () => writeFileSync(SEEN_FILE, JSON.stringify([...seen].slice(-8)));

// ----- Discord -----
const COLORS = { vinted: 0x09b1ba, ebay: 0xe53238 } as const;
async function sendToDiscord(l: Listing, searchName: string, retry = 0): Promise<void> {
  const url = process.env.DISCORD_WEBHOOK_URL;
  if (!url) throw new Error("DISCORD_WEBHOOK_URL manquant (variable Railway)");
  const body = {
    username: "Phone Scanner",
    embeds: [{
      title: l.title.slice(0, 250),
      url: l.url,
      color: COLORS[l.source],
      fields: [
        { name: "💰 Prix", value: `**${l.price} ${l.currency}**`, inline: true },
        { name: "🛒 Source", value: l.source.toUpperCase(), inline: true },
        { name: "🔍 Recherche", value: searchName, inline: true },
        ...(l.extra ? [{ name: "ℹ Infos", value: l.extra, inline: false }] : [])
      ],
      ...(l.image ? { image: { url: l.image } } : {}),
    ]
  };
  await fetch(url, { method: "POST", headers: { "Content-Type": "application/json" }, body: JSON.stringify(body) });
}

```

```

        timestamp: new Date().toISOString(),
    }],
};
const r = await fetch(url, {
    method: "POST",
    headers: { "content-type": "application/json" },
    body: JSON.stringify(body),
});
if (r.status === 429 && retry < 3) {
    const j: any = await r.json().catch(() => ({}));
    await sleep(((j.retry_after ?? 2) as number) * 1000 + 200);
    return sendToDiscord(l, searchName, retry + 1);
}
if (!r.ok) throw new Error(`Discord HTTP ${r.status}`);
}

// ----- Vinted -----
const UA = "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML,
const VBASE = "https://www.vinted.fr";
let cookie = "";
let cookieAt = 0;
async function refreshCookie() {
    const r = await fetch(VBASE + "/", { headers: { "user-agent": UA, "accept-langu
    cookie = r.headers.getSetCookie().map((c) => c.split(";")[0]).join("; ");
    cookieAt = Date.now();
}
const vHeaders = () => ({ "user-agent": UA, accept: "application/json, text/plain

async function searchVinted(query: string, maxPrice?: number): Promise<Listing[]>
    if (!cookie || Date.now() - cookieAt > 20 * 60_000) await refreshCookie();
    const u = new URL(VBASE + "/api/v2/catalog/items");
    u.searchParams.set("search_text", query);
    u.searchParams.set("order", "newest_first");
    u.searchParams.set("per_page", "30");
    u.searchParams.set("page", "1");
    if (maxPrice) u.searchParams.set("price_to", String(maxPrice));
    let r = await fetch(u, { headers: vHeaders() });
    if (r.status === 401 || r.status === 403) {
        await refreshCookie();
        r = await fetch(u, { headers: vHeaders() });
    }
    if (!r.ok) throw new Error(`Vinted HTTP ${r.status}`);
    const data: any = await r.json();
    return (data.items ?? []).map((it: any): Listing => {
        const isObj = typeof it.price === "object";
        return {
            id: `vinted:${it.id}`,

```

```

        source: "vinted",
        title: it.title,
        price: isObj ? Number(it.price.amount) : Number(it.price),
        currency: isObj ? it.price.currency_code : it.currency ?? "EUR",
        url: it.url ?? `${VBASE}${it.path}`,
        image: it.photo?.url,
        extra: [it.brand_title, it.size_title, it.status].filter(Boolean).join(" ·
    });
});
}

// ----- eBay (optionnel) -----
let token = "";
let tokenExp = 0;
async function ebayToken(): Promise<string> {
    if (token && Date.now() < tokenExp) return token;
    const auth = Buffer.from(`${process.env.EBAY_CLIENT_ID}:${process.env.EBAY_CLIENT_SECRET}`);
    const r = await fetch("https://api.ebay.com/identity/v1/oauth2/token", {
        method: "POST",
        headers: { "content-type": "application/x-www-form-urlencoded", authorization: `Basic ${auth.toString('base64')}` },
        body: "grant_type=client_credentials&scope=https%3A%2F%2Fapi.ebay.com%2Foauth2%2Fclient_credentials"
    });
    if (!r.ok) throw new Error(`eBay token HTTP ${r.status}`);
    const j: any = await r.json();
    token = j.access_token;
    tokenExp = Date.now() + (j.expires_in - 120) * 1000;
    return token;
}

async function searchEbay(query: string, maxPrice?: number): Promise<Listing[]> {
    if (!process.env.EBAY_CLIENT_ID || !process.env.EBAY_CLIENT_SECRET) return [];
    const u = new URL("https://api.ebay.com/buy/browse/v1/item_summary/search");
    u.searchParams.set("q", query);
    u.searchParams.set("sort", "newlyListed");
    u.searchParams.set("limit", "30");
    const filters = ["priceCurrency:EUR"];
    if (maxPrice) filters.unshift(`price:[..${maxPrice}]`);
    u.searchParams.set("filter", filters.join(","));
    const r = await fetch(u, {
        headers: { authorization: `Bearer ${await ebayToken()}`, "X-EBAY-C-MARKETPLACE": process.env.EBAY_MARKETPLACE }
    });
    if (!r.ok) throw new Error(`eBay HTTP ${r.status}`);
    const data: any = await r.json();
    return (data.itemSummaries ?? []).map((it: any): Listing => ({
        id: `ebay:${it.itemId}`,
        source: "ebay",
        title: it.title,
        price: Number(it.price?.value),

```

```

        currency: it.price?.currency ?? "EUR",
        url: it.itemWebUrl,
        image: it.image?.imageUrl,
        extra: [it.condition, it.itemLocation?.country].filter(Boolean).join(" · ") |
    )));
}

// ----- Boucle principale -----
const allowed = (l: Listing, s: Search) =>
    ![...config.exclude, ...(s.exclude ?? [])].some((w) => l.title.toLowerCase().in

async function cycle(silent: boolean) {
    for (const s of config.searches) {
        const sources = [
            { name: "vinted", run: () => searchVinted(s.query, s.maxPrice) },
            { name: "ebay", run: () => searchEbay(s.query, s.maxPrice) },
        ];
        for (const src of sources) {
            try {
                const items = (await src.run()).reverse();
                for (const l of items) {
                    if (seen.has(l.id)) continue;
                    seen.add(l.id);
                    if (silent || !allowed(l, s)) continue;
                    await sendToDiscord(l, s.name);
                    console.log(`➡️ ${l.source} | ${l.price}€ | ${l.title}`);
                    await sleep(1200);
                }
            } catch (e) {
                console.error(`[${src.name}] "${s.query}" :`, (e as Error).message);
            }
            await sleep(1500);
        }
    }
    saveSeen();
}

console.log("📞 Phone Scanner démarré");
try {
    await sendToDiscord(
        { id: "start", source: "vinted", title: "✅ Phone Scanner est en ligne", price: 0,
        "Démarrage"
    );
} catch (e) {
    console.error("Discord :", (e as Error).message);
}

```

```
await cycle(wasEmpty);  
while (true) {  
    await sleep(config.intervalSeconds * 1000);  
    await cycle(false);  
}
```